

EC327 Final Cheat Sheet

Object-Oriented C++ - Polymorphism - Exceptions - Templates - STL - Algorithms

Design principle: concept-first, not practice-exam-copying. Use this as a printed handbook for solving new final-style coding problems.

How to use this packet

- Start with compile-first templates: most final problems are provided-code modifications.
- When stuck, identify the pattern: encapsulation, inheritance, exception, dynamic cast/operator, template array, STL/data structure, or algorithm.
- Use the templates as syntax anchors, then adapt variable/class names to the exam prompt.

Concept weighting from the practice finals

Weight	Topics	Why it matters
Very high	Encapsulation, inheritance, virtual functions, abstract classes, <code>dynamic_cast</code>	Core OOP/polymorphism questions appear repeatedly.
High	Exceptions, custom exception classes, static members, destructors	Final-style questions use throw/catch and destructor behavior.
High	Templates + arrays/pointers	Generic T functions with dynamic arrays and size-by-reference.
Medium	STL / data structures / input validation	Could be stack-specific or open-ended stateful coding.
Medium	Operator overloading	Usually tied to polymorphism and dynamic casting.
Low-medium	Sorting, Big-O, function pointers	Likely supporting concept or smaller section.

Fast exam rule

If you cannot finish functionality, leave compiling, documented code. A compiling partial solution is far better than a beautiful broken one.

0. Compile / Submit / Debug Survival

Compile patterns

```
# one file
g++ Q1.cpp -o q1
./q1

# multiple files
g++ main.cpp Point2D.cpp Vector2D.cpp GameObject.cpp -o
prog

# compile only to object file
g++ -c Point2D.cpp

# common with C++11
g++ -std=c++11 *.cpp -o prog
```

Zip pattern

```
mkdir username_5557_002_final
cd username_5557_002_final
# put Q1.cpp ... Q7.txt here
cd ..
zip -r username_5557_002_final.zip
./username_5557_002_final/*
less username_5557_002_final.zip # q to quit
```

Error quick map

Error	Usually means
undefined reference	Linker cannot find a definition. Compile all .cpp files or define function.
undeclared	Name not declared, misspelled, or out of scope.
no matching function	Arguments/prototype mismatch.
cannot access private	Need getter/setter/friend/member function; don't change access unless allowed.
abstract type	Class still has an unimplemented pure virtual function.
bad_cast	Failed dynamic_cast by reference.

Exam workflow

- Compile provided code before editing.
- Uncomment one block at a time, compile after each block.
- Never change main if prompt says not to; add/fix class/functions around it.
- Prefer minimal changes to base classes: add virtual, getter/setter, friend, or constructor only as needed.
- Use exact filenames, cases, prototypes, and parameter types from prompt.

1. Class Syntax, Encapsulation, and this

class vs struct

Construct	Default access	Use
class	private	Normal OOP classes with encapsulation.
struct	public	Plain data or simple records.

Basic class template

```
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    bool studied;
    int iq;
protected:
    int classesTaken;
public:
    Student() : studied(false), iq(100), classesTaken(0) {}

    bool getStudied() const { return studied; }
    int getIq() const { return iq; }
    void setStudied(bool s) { studied = s; }
};
```

this pointer

- Inside a non-static member function, **this** is a pointer to the current object.
 - **this->x** means the same as **(*this).x**.
 - Use it when parameter names shadow member names: **this->iq = iq;**
- ```
class A {
 int x;
public:
 void setX(int x) { this->x = x; }
 A* self() { return this; }
};
```

# 2. Access Control, Getters/Setters, Friend

| Access    | Outside class | Derived class | Same class/friend |
|-----------|---------------|---------------|-------------------|
| public    | yes           | yes           | yes               |
| protected | no            | yes           | yes               |
| private   | no            | no            | yes               |

## When access is private/protected

- Outside code cannot directly touch private/protected members.
- Use member getter/setter or friend function/class if the prompt asks for non-member access.
- Do not change access level when the exam says access level must stay the same.

## Friend function/class

```
class Superhero {
private:
 int iq;
protected:
 double strength;
public:
 friend int iq_getter(Superhero s);
 friend void iq_setter(Superhero* s, int val);
 friend class Ironman;
};

int iq_getter(Superhero s) { return s.iq; }
void iq_setter(Superhero* s, int val) { s->iq = val; }
```

**Friend is not a member. It is an outside function/class granted access to private/protected data.**

# 3. Constructors, Destructors, Static Members

## Constructor patterns

```
class Box {
private:
 int width;
 string label;
public:
 Box() : width(0), label("none") {}
 Box(int w, string s) : width(w), label(s) {}
};
```

- Initialization list starts with **:** after the parameter list.
- Constructors have no return type and same name as class.
- Default constructor takes no arguments.
- Parameterized constructors can be overloaded.

## Derived constructor calls base constructor

```
class Videogame {
private:
 int cheatcode;
public:
 Videogame(int c) : cheatcode(c) {}
};
```

```
class DonkeyKong : public Videogame {
public:
 DonkeyKong(int c) : Videogame(c) {}
};
```

## Destructor + virtual destructor

```
class Base {
public:
 virtual ~Base() { cout << "Base deleted
"; }
};
class Derived : public Base {
public:
 ~Derived() { cout << "Derived deleted
"; }
};
```

```
Base* p = new Derived;
delete p; // needs virtual destructor for correct cleanup
```

## Constructor/destructor order

| Event                          | Order                                      |
|--------------------------------|--------------------------------------------|
| Create derived object          | base constructor, then derived constructor |
| Destroy derived object         | derived destructor, then base destructor   |
| Local objects during exception | destructors run during stack unwinding     |

## Static member variable

```
class Student {
public:
 static int objects;
 Student() { objects++; }
 ~Student() { objects--; }
};

int Student::objects = 0; // define/init in global area
```

**Static data member belongs to the class, not one object. It must be defined once outside the class.**

# 4. Copying, Shallow vs Deep Copy

## Shallow copy danger

- Default copy copies pointer values, not heap data.

- Two objects may point to the same heap array/string.
- Deleting one object can leave the other with a dangling pointer.

### Deep copy skeleton

```
class Course {
private:
 string* students;
 int size;
public:
 Course(const Course& other) {
 size = other.size;
 students = new string[size];
 for (int i=0; i<size; i++)
 students[i] = other.students[i];
 }
 ~Course() { delete [] students; }
};
```

- If class owns heap memory, think Rule of Three: destructor, copy constructor, assignment operator.
- Pass large objects by const reference to avoid unnecessary copy: void f(const Course& c).

## 5. Inheritance Essentials

### is-a relationship

```
class Student {
public:
 virtual void setStudied(bool s) { studied = s; }
protected:
 bool studied;
};

class Slacker : public Student {
public:
 void setStudied(bool s) override { studied = false; }
};
```

- Derived class is a specialized version of base class.

- Public inheritance models is-a: `Slacker` is a `Student`.

- Base private members exist inside derived object but cannot be directly accessed by derived methods.

### Not inherited

| Not inherited from base | Reason                       |
|-------------------------|------------------------------|
| constructors            | derived has own construction |
| destructor              | derived has own destruction  |
| copy constructor        | special member function      |
| assignment operator     | special member function      |

### Access under public inheritance

| Base member | As seen by derived | As seen outside |
|-------------|--------------------|-----------------|
| public      | public             | public          |
| protected   | protected          | not direct      |
| private     | not direct         | not direct      |

### has-a vs is-a

```
class Engine { double cc; };
class Motorcycle {
 Engine objEng; // has-a: composition/aggregation
};
```

```
class DonkeyKong : public Videogame { }; // is-a
```

## 6. Virtual Functions and Polymorphism

**Polymorphism requires: inheritance + virtual function + base pointer/reference to a derived object.**

### Static binding problem

```
class Base { public: void f(){ cout << "Base"; } };
class D1 : public Base { public: void f(){ cout << "D1"; }
};
```

```
D1 d;
Base* p = &d;
p->f(); // Base::f if f is NOT virtual
```

### Dynamic binding solution

```
class Base { public: virtual void f(){ cout << "Base"; } };
class D1 : public Base { public: void f() override { cout <<
"D1"; } };
```

```
D1 d;
Base* p = &d;
p->f(); // D1::f because f is virtual
```

### Base pointer/reference rules

| Expression                           | Meaning                           |
|--------------------------------------|-----------------------------------|
| <code>Base* p = new Derived;</code>  | allowed upcast                    |
| <code>p-&gt;virtualFunc();</code>    | calls derived override            |
| <code>p-&gt;nonVirtualFunc();</code> | calls base version                |
| <code>delete p;</code>               | base destructor should be virtual |

## 7. Abstract Classes and Interfaces

### Pure virtual function

```
class Student {
public:
 virtual bool getStudied() = 0; // pure virtual
 virtual ~Student() {}
};
```

```
class Slacker : public Student {
public:
 bool getStudied() override { return false; }
};
```

- `= 0` makes function pure virtual.

- Class with at least one pure virtual function is abstract.

- Cannot instantiate abstract base class: `Student s;` is invalid.

- Can use abstract base pointers/references: `Student* p = new Slacker;`

### const virtual functions

```
class Shape {
public:
 virtual double getArea() const = 0;
};
class Circle : public Shape {
public:
 double getArea() const override { return 3.14*r*r; }
private:
 double r;
};
```

- `const` after parameter list means member function does not modify object.

- Derived override must match the `const`: `getArea() const`.

### Abstract class vs concrete class

| Class            | Can instantiate? | Typical role                  |
|------------------|------------------|-------------------------------|
| abstract base    | no               | interface/common pointer type |
| derived concrete | yes              | actual object behavior        |

## 8. Runtime Type: `dynamic_cast`, `static_cast`, `typeid`

### `dynamic_cast` pointer

```
Student& s = ...;
Slacker* sp = dynamic_cast<Slacker*>(&s);
if (sp != NULL) {
 // s is actually a Slacker
} else {
 // not a Slacker
}
```

### `dynamic_cast` reference

```
try {
 Slacker& sr = dynamic_cast<Slacker&>(studentRef);
}
catch (bad_cast& e) {
 cout << "not a Slacker";
}
```

| Cast                                                   | Safe?               | Use                                                |
|--------------------------------------------------------|---------------------|----------------------------------------------------|
| <code>dynamic_cast&lt;Derived*&gt;(basePtr)</code>     | runtime checked     | polymorphic downcast; returns NULL/nullptr if fail |
| <code>dynamic_cast&lt;Derived&amp;&gt;(baseRef)</code> | runtime checked     | throws <code>bad_cast</code> if fail               |
| <code>static_cast&lt;Derived*&gt;(basePtr)</code>      | not runtime checked | only when you are sure                             |
| <code>typeid(obj).name()</code>                        | runtime type info   | debug / type display                               |

**`dynamic_cast` needs a polymorphic base class, usually at least one virtual function.**

## 9. Operator Overloading for Objects

### Member vs non-member

| Kind       | Syntax          | Private access?  | When useful                         |
|------------|-----------------|------------------|-------------------------------------|
| member     | A::operator+(B) | yes              | LHS is this object                  |
| non-member | operator+(A,B)  | no unless friend | symmetric ops; final practice style |

### Rules that never change

- Cannot change operator precedence, associativity, or arity.
- Overload should preserve intuitive meaning; don't surprise the grader.
- If returning heap object, caller must eventually delete it unless exam ignores cleanup.

### Polymorphic operator template

```
Student* operator+(Student& a, Student& b)
{
 if (dynamic_cast<Slacker*>(&a) != NULL ||
 dynamic_cast<Slacker*>(&b) != NULL)
 return new Slacker();
 return new MedStudent();
}

Student* operator*(Student& a, Student& b)
{
 if (dynamic_cast<Slacker*>(&a) != NULL &&
 dynamic_cast<Slacker*>(&b) != NULL)
 return new Slacker();
 return new MedStudent();
}

Student* operator/(Student& numerator, Student& denominator)
{
 if (dynamic_cast<Slacker*>(&denominator) != NULL)
 return new Slacker();
 return new MedStudent();
}
```

### Member operator example

```
class Whale {
public:
 double length;
 whale operator/(Whale& other) {
 whale result;
 result.length = length / other.length;
 return result;
 }
};
```

## 10. Object Passing and References

| Parameter        | Copies object? | Can modify caller? | Use                              |
|------------------|----------------|--------------------|----------------------------------|
| Student s        | yes            | no                 | small objects only; slicing risk |
| Student& s       | no             | yes                | polymorphism / modify object     |
| const Student& s | no             | no                 | read-only, efficient             |
| Student* s       | no             | yes via ->         | can be NULL; heap/polymorphism   |

**Avoid object slicing: passing Derived by value to a Base parameter loses derived-specific part. Use Base& or Base\* for polymorphism.**

## 11. Exceptions: try, throw, catch

### Basic pattern

```
try {
 if (num2 == 0)
 throw num1; // throws an int
 cout << num1 / num2;
}
catch (int ex) {
 cout << ex << " cannot be divided by 0";
}
```

- Code after throw inside the try block is skipped.
- catch(int ex) catches only thrown int values.
- If no matching catch exists, exception propagates up the call stack.

### Three patterns

| Pattern | Where throw happens                           | Where catch happens  |
|---------|-----------------------------------------------|----------------------|
| 1       | inside same try block                         | same function        |
| 2       | called function                               | caller function      |
| 3       | called function catches locally then rethrows | caller catches final |

### Pattern 2: throw in called function

```
int quotient(int a, int b) {
 if (b == 0) throw 0;
 return a / b;
}

try { cout << quotient(x,y); }
```

```
catch (int ex) { cout << "division error"; }
```

### Pattern 3: rethrow

```
void f() {
 try {
 throw 7;
 }
 catch (int ex) {
 // local cleanup/message
 throw; // rethrow same exception upward
 }
}
```

## 12. Custom Exception Classes

### Derived from exception

```
#include <exception>
using namespace std;

class StudiesException : public exception {
public:
 Student* sPtr;
 StudiesException(Student* s) : sPtr(s) {
 cout << this->what() << ": ";
 }
};
```

### Throw from member function

```
class Slacker : public Student {
public:
 void setStudied(bool input) override {
 throw StudiesException(this);
 }
};
```

### Catch

```
try {
 p->setStudied(true);
}
catch (StudiesException& ex) {
 cout << "cannot study" << endl;
 cout << ex.sPtr->getIq() << endl;
}
```

| Catch style                | Comment                             |
|----------------------------|-------------------------------------|
| catch (E ex)               | copies exception                    |
| catch (E& ex)              | preferred if modifying/large object |
| catch (const exception& e) | common for standard exceptions      |
| catch (...)                | generic catch, must be last         |

## 13. Stack Unwinding and Destructor Traps

### What happens when exception is thrown

- Control leaves current try block/function until a matching catch is found.
- Local fully constructed objects are destroyed while unwinding.
- Destructors should not throw; throwing during unwinding can terminate the program.

### Constructor throwing example

```
class A {
public:
 A(int n) : m_n(n) {
 if (n == 0) throw logic_error("0");
 }
 ~A() { cout << m_n; }
 const int m_n;
};

try {
 A a(1); // constructed, destructor runs on unwind
 A b(0); // throws; b destructor does not run
 A c(2); // never constructed
}
catch (const exception&) { cout << 3; }
```

### Generic catch

```
try { risky(); }
catch (int x) { /* int only */ }
catch (double d) { /* double only */ }
catch (...) { /* anything else; put last */ }
```

### Exception vs traditional error checking

| Approach        | Use                                         |
|-----------------|---------------------------------------------|
| if checks       | simple local validation                     |
| return sentinel | OK if return value not overloaded/confusing |
| throw/catch     | separate error path from productive code    |

## 14. Template Functions and Generic Type T

### T means placeholder type

```
template <typename T>
T maxValue(T a, T b)
{
 return (a > b) ? a : b;
}

maxValue(2, 5); // T = int
maxValue(2.2, 5.1); // T = double
```

- typename and class are equivalent in template parameter lists.
- The operations used inside must exist for T: +, <, ==, etc.
- Template definitions usually live in header files, because compiler needs full definition for each T.

### Template class skeleton

```
template <typename T>
class Stack {
private:
 T* elements;
 int size;
public:
 Stack() : size(0) { elements = new T[100]; }
 ~Stack() { delete [] elements; }
 void push(T v) { elements[size++] = v; }
 T top() const { return elements[size-1]; }
};
```

## 15. Template Arrays: destroy / replace Pattern

### Destroy: remove all target values

```
template <typename T>
T* destroy(T list[], int listSize, int& destroyedSize, T
target)
{
 int count = 0;
 for (int i=0; i<listSize; i++)
 if (list[i] != target) count++;

 destroyedSize = count;
 T* result = new T[destroyedSize];

 int j = 0;
 for (int i=0; i<listSize; i++)
 if (list[i] != target)
 result[j++] = list[i];

 return result;
}
```

### Replace: same size, substitute target

```
template <typename T>
T* replace(T list[], int listSize, T insert, T target)
{
 T* result = new T[listSize];
 for (int i=0; i<listSize; i++) {
 if (list[i] == target)
```

```
 result[i] = insert;
 else
 result[i] = list[i];
 }
 return result;
}
```

### Why int& destroyedSize?

- Function must tell caller the size of the new returned array.
- Return value is already used for T\*, so size is updated by reference.
- Caller uses: T\* p = destroy(list, n, newSize, target);

### Dynamic memory rules

| Allocate    | Free                                        |
|-------------|---------------------------------------------|
| new T       | delete p                                    |
| new T[n]    | delete [] p                                 |
| returned T* | caller owns it unless prompt says otherwise |

## 16. Arrays and Pointer Arithmetic Review

### Heap array access

```
int* p = new int[2];
p[0] = 10;
p[1] = 20;
cout << p[0]; // 10
cout << *(p + 1); // 20
delete [] p;
```

### Pointer arithmetic addresses

- p + k moves by k elements, not k bytes.
- If double\* starts at address 1002500, then p+8 is 1002500 + 8\*8 = 1002564.

## 17. STL Core: Containers, Iterators, Algorithms

### Four STL components

| Component        | Meaning                                         |
|------------------|-------------------------------------------------|
| Containers       | hold data: vector, list, deque, map, set, stack |
| Iterators        | fancy pointers into containers                  |
| Algorithms       | sort, find, count, copy, etc.                   |
| Function objects | callable objects used by algorithms             |

### Container cheat table

| Container  | Shape              | Key ops                        |
|------------|--------------------|--------------------------------|
| array<T,N> | fixed array        | size(), operator[]             |
| vector<T>  | dynamic array      | push_back, pop_back, erase     |
| deque<T>   | front/back dynamic | push_front/back                |
| list<T>    | linked list        | insert/erase, no random access |
| stack<T>   | LIFO adapter       | push, top, pop, empty          |
| queue<T>   | FIFO adapter       | push, front, pop               |
| map<K,V>   | key-value          | m[key], find                   |

### Iterator basics

```
vector<int> v {1,2,3,4};
vector<int>::iterator it = v.begin();
cout << *it; // first element
cout << *(it+2); // random-access iterator only
```

### Algorithms

```
#include <algorithm>
```

```
sort(v.begin(), v.end());
auto it = find(v.begin(), v.end(), target);
if (it != v.end()) cout << "found";
```

## 18. STL stack + Call Stack Pattern

### Stack essentials

```
#include <stack>
stack<Function> callStack;

callStack.push(Function("foo", 10));
Function f = callStack.top(); // top does not remove
callStack.pop(); // pop removes, returns void
bool empty = callStack.empty();
```

### Exam template

```
void call(string func_name, int val)
{
 callStack.push(Function(func_name, val));
}

int ret()
{
 int v = callStack.top().getVal();
 callStack.pop();
 return v;
}

void clearStack()
{
 while (!callStack.empty()) {
 cout << callStack.top().getName() << endl;
 callStack.pop();
 }
}
```

**pop() does not return the removed element. Save top() first, then pop().**

## 19. Open-Ended Data Structure / State Coding

### If final asks for a small game/simulation

- Create state variables first: sum, counts array, boolean loop flag, validity flag.
- Validate input before modifying state.
- Use helper functions to keep main readable if allowed.
- Match sample output exactly: no extra cout messages.

### Input validation skeleton

```
int count[15] = {0};

bool validCard(int card) {
 return card >= 2 && card <= 14;
}

bool stillAvailable(int card) {
 return count[card] < 4;
}

int cardValue(int card, int currentSum) {
 if (card >= 11 && card <= 13) return 10;
 if (card == 14) return (currentSum <= 10 ? 11 : 1);
 return card;
}
```

### Loop pattern

```
bool running = true;
while (running) {
 cin >> x;
 if (!valid(x)) { cout << "Invalid Entry
"; continue; }
 update_state(x);
 if (win()) running = false;
 else if (lose()) running = false;
}
```

## 20. Function Pointers

### Function vs pointer-to-function

```
int foo(); // function prototype
int* pFoo(); // function returning int*
int (*ptrFoo)(); // pointer to function returning int
```

```
int hoo(int x);
int (*phoo)(int) = hoo;
hoo(10);
phoo(10);
```

### Passing function as argument

```
int compareAsc(int a, int b);

void specialSort(int arr[], int n, int (*cmp)(int,int))
{
 // call cmp(arr[i], arr[j]) inside sort
}

specialSort(a, n, compareAsc);
```

## 21. Algorithms and Big-O

### Big-O facts

| Complexity | Meaning                            |
|------------|------------------------------------|
| O(1)       | constant time                      |
| O(log n)   | halves search space                |
| O(n)       | one pass                           |
| O(n log n) | efficient comparison sort          |
| O(n^2)     | nested loops / pairwise comparison |

### Sorting

| Sort   | Best/Avg   | Worst      | Notes                                 |
|--------|------------|------------|---------------------------------------|
| Bubble | O(n^2)     | O(n^2)     | simple adjacent swaps                 |
| Merge  | O(n log n) | O(n log n) | uses extra memory; stable idea        |
| Quick  | O(n log n) | O(n^2)     | often fast in practice; pivot matters |
| Heap   | O(n log n) | O(n log n) | uses heap; easy priority behavior     |

### Bubble sort comparison count

$(n-1) + (n-2) + \dots + 1 = n(n-1)/2 \Rightarrow O(n^2)$

### Merge sort idea

- Divide array into halves until size 1.
- Merge sorted halves back together.
- Time  $O(n \log n)$  because  $\log n$  levels and  $O(n)$  work per level.

## 22. Multiple-Choice Concept Traps

| Question type              | Answer anchor                                     |
|----------------------------|---------------------------------------------------|
| const reference            | prevents modification of referenced value         |
| p = q                      | p now points where q points; no int values copied |
| member variables/functions | state / behavior                                  |
| private base members       | not directly accessible in derived class          |
| multiple inheritance       | derived class has 2+ base classes                 |
| constructor order          | base first, derived second                        |
| destructor order           | derived first, base second                        |
| has-a                      | composition: object contains another object       |
| operator overload limits   | cannot change precedence, associativity, arity    |

## 23. Final-Style Pattern Recognition

### Encapsulation question

- Find private/protected/public variables.
- Add getters/setters or friends as required.
- Do not change access if forbidden.
- Uncomment main; fix only typos if allowed.

### Inheritance question

- Create derived class with : public Base.
- Call base constructor in initialization list if base private data must be initialized.
- Use virtual in base when main uses base pointer/reference and needs derived behavior.

### Exception question

- Create exception class derived from exception.
- Store pointer/reference to object if prompt says exception should know which object failed.
- Throw in derived override; catch in main or caller.

### Operator + dynamic cast question

- Make base abstract if given pure virtual function.
- Use base reference parameters: Student&.
- Use dynamic\_cast<Derived\*>(&obj) to test actual type.
- Return new Slacker() or new MedStudent() as Student\*.

### Template array question

- Use template<typename T> before function.
- Allocate new array with new T[size].
- Use reference parameter to return new size.

### Data structure question

- Know stack, vector, arrays, count arrays, validation loops.

- Sample output controls cout; avoid extra prints.

## 24. Header / CPP Split for Classes

### Header file

```
#ifndef CIRCLE_H
#define CIRCLE_H

class Circle {
private:
 double radius;
public:
 Circle();
 Circle(double r);
 double getArea() const;
};

#endif
```

### CPP file

```
#include "Circle.h"

Circle::Circle() : radius(1.0) {}
Circle::Circle(double r) : radius(r) {}

double Circle::getArea() const {
 return 3.14 * radius * radius;
}
```

**Do not redeclare the entire class in the .cpp. Include the header and define functions with ClassName::functionName.**



## 25. Mini Example Bank

### Accessor/mutator with protected derived access

```
class Base {
private:
 int secret;
protected:
 int semiSecret;
public:
 int getSecret() const { return secret; }
 void setSecret(int v) { secret = v; }
};
class Derived : public Base {
public:
 void f() { semiSecret = 10; /* secret = 5; invalid */ }
};
```

### Virtual override through base pointer

```
class A { public: virtual void print(){ cout << "A"; } };
class B : public A { public: void print() override { cout <<
 "B"; } };
```

```
A* p = new B;
p->print(); // B
```

### Friend class

```
class Square;
class Rectangle {
private:
 int width, height;
public:
 friend class Square;
};
class Square {
public:
 void steal(Rectangle& r) { r.width = 10; }
};
```

### Member access pointer equivalence

```
b->isitme(a);
(*b).isitme(a); // same meaning; parentheses required
```

### const char\* vs string

```
const char* decode(int x) { return "Fixed text"; }
```

```
#include <string>
string decode2(int x) { return "Fixed text"; }
```

## 26. Last-Minute Checklist

| Before submit | Check                                                                   |
|---------------|-------------------------------------------------------------------------|
| Files         | Q1.cpp ... Q7.txt in the folder; no missing required files              |
| Compilation   | Each coding question compiles separately                                |
| Main          | Did not change main when forbidden                                      |
| Virtual       | Base function virtual if derived override must run through base pointer |
| Destructor    | Base destructor virtual when deleting derived via base pointer          |
| Casts         | dynamic_cast pointer checked against NULL/nullptr                       |
| Exceptions    | Matching catch type exists                                              |
| Templates     | template line present; T operations legal                               |
| Memory        | new[] paired with delete[]; returned arrays have size info              |
| Output        | No extra cout if sample output must match                               |